2009 International Conference on Computational Science and Engineering

Implementation of Unlimited Integer

Kong Weiguang[1], Zhu Yong[1], Wang Yuan[2]
[1]College of Computer Science, Wuhan University of Science and Engineering, Wuhan, 430074,P.R.China
[2]Library, Zhongnan University of Economics and Law, Wuhan, 430074,P.R.China
wgkong@public.wh.hb.cn

***Abstract*—This paper analyses the expression of digital symbol and calculation of number system, and develops 2^n based number system whose digital symbol can be easily expressed with built-in data type variable in high-level programming language. By researching the characteristic of scalable queue templates in visual C++ template library, it gives the implementation scheme of unlimited integer class based on automatic scalable queue. In this scheme, one unlimited integer is made up by one queue, each element of the queue is one digital of the number. This paper also researches the algorithm of arithmetic calculations and tests its performance in RSA cryptogram of 1024 or more bits. The result indicates that this numeric class is practicable, and it can be a member of base library of high level programming language.**

***Index Terms*—Number System, Digital Symbol, Unlimited Integer, Queue.**

I. INTRODUCTION

Big integer data is useful in high precision applications which calculate between big numbers from hundreds digits to thousands digits, such as applications in the field of compute algebra, geometric reasoning, cryptogram and microcosmic simulation. In digital data encryption, large primes and integers are often used, for example, RSA algorithm can be used to encrypt data and authenticate identification, its security is founded on the difficulty of large integer factorization, and it has resisted kinds of attacks successfully and kept valuable. Larger number is requisite as the developing of computer technology.

Each high-level programming language defines its built-in data types, such as int, long, _int64, char and string in C++. Simple operations can be effectively performed by built-in data type variables. It is a pity that all built-in data type are limited, for instance, in C++, number bigger than the maximum integer of type _int64 must be turned to float type which is imprecise. It is necessary to develop a special data type to accurately express numbers larger than maximum integer or with much more decimal digits.

One solution is treating large number as a string made up of decimal digital symbols, and all arithmetic calculations including addition, subtraction, multiplication and division are turned to operations of chars in the strings in term of manual work pattern. For instance, a 1024-bit integer may be turned to a decimal string with hundreds of chars. The strongpoint of this solution is that it is easy to be understood and settles the overflow puzzle of large number, but it has two shortcomings: the first is that simulant calculation between chars has poorer efficiency than built-in integer types, the second is that it will take long time to loop on long strings of decimal digit symbols.

Another solution is treating big number as a stream of binary bits, all arithmetic calculations are based on shift and logic operation of binary bits. This solution has high space efficiency, but each operation on binary bits will take long time, and the program code is complex and unreadable. In real projects, a specified array is generally used to express big number and achieve calculations of big number.

In projects founded on limited array, it is necessary to compile another similar class based on a longer array when digits of big number need to be expanded, and it is troublesome to operate between classes with different length arrays because they are different class.

Reference [1] researched the relation between operate speed and number system basement, and concluded that the operate speed is faster when the number system basement is bigger。 Reference [2] designed a big number of 2^{32} based number system, it turned a 1024 bit number to an array with 32 elements of 2^{32} number system, and calculate imitating human's pattern. Reference [3] advanced a scheme of big number on chars, constructed a calculation system of big number and analyzed its performance. References [4-13] researched kinds of calculations of big integers separately.

The solution of big number in this paper is based on scalable queue, the number is in 2^n based number system, and it is unlimited and can be computed efficiently.

II. NUMBER SYSTEM AND SYMBOL

In the real world, common used numbers without special statement are decimal. There are ten symbols (0,1,…,9) in decimal number system, its radix is 10. The numerical size is the sum of products of every digital multiplies its right of the position. Different positions have different rights. For example, the value of integer 123 can be computed through the formula:

$$123=1\cdot10^2+2\cdot10^1+3\cdot10^0 \quad (1)$$

It is known that data in the computer is binary finally. The radix of binary number system is 2. Other number systems are familiar including hexadecimal and octal. The value of one number in different number system is unaltered.

By the same token, M based number system can be defined as following:

1, There are M symbols from 0 to M-1 in the system;
2, The base radix is M.

For example, one number in M based number system can be computed as formula:

978-0-7695-3823-5/09 $26.00 © 2009 IEEE
DOI 10.1109/CSE.2009.196

IEEE computer society

$$B_2B_1B_0=B_2\cdot M^2+B_1\cdot M^1+B_0\cdot M^0 \quad (2)$$

In computer, it is necessary to figure each digital symbol of M based number system in order to implement its operations. If M equals 2^n, it is suitable to figure M digital symbols with different combination of n binary bits. For instance, in octal number system, symbols 0~7 can be represent with combination of three binary bits: 000, 001, 010, 011, 100, 101, 110, and 111. In other words, three binary bits can be treated as a digital symbol of octal number system. If n is 8, it is possible to use a byte to figure the digital symbols of 2^8 based number system, and if n is 16, it is suitable to use a word to figure the digital symbols of 2^{16} based number system, and if n is 32, it is appropriate to use four bytes to figure the digital symbols of 2^{32} based number system, and so on.

The arithmetic and logic operation of numbers in all number systems is actually operated on its digital symbols. In order to save digital symbols of M based number system expediently, it is the best to figure each digital symbol with a variable of built-in data type (byte, int, long, _int64 etc.) in high-level programming language because the basic operations of these types have been programmed efficiently. Considering that the length of operation result of two digital may be 1 or 2 digitals, the principle to define suitable n is that the longest built in type of current high level language can hold two digital of 2^n based number system, namely, n does not exceed half of the length of the maximum integer type.

Number of M based number system can be organized with a queue of digitals. The queue is scalable. The queue and a Boolean variable represent the sign of number form the basic data structure of unlimited integer class.

III. DEQUE IN C++

C++ template class library is part of stand C++ language; it has defined perfect data containers, usable iterators and many efficient algorithms. Because C++ standard template library is expandable and the containers are scalable, these templates can construct classes with many more functions.

There are kinds of scalable queues in C++ template class, they are deque, vector and list, they all support random access of all elements, and each of them has its own specialty. Vector is good at efficient insertion and deletion at the end of queue and the operation time at the head or in the middle of queue is linear. Unlikely, deque's time of operating on random position of queue is efficient and invariable. By comparison, deque has the advantage of array and list at the same time, it can be operated as convenient as array and can flex to meet special need.

Deque is suitable to be the container of digital queue of unlimited numeric class. It will be convenient to insert or delete elements at the head or end of digital queues, and it is easy to access any element randomly. More importantly, the count of elements in deque is unlimited. In visual C++, commonly used operation function of deque is listed as following:

begin()	Return iterator pointing to the head of queue;
end()	Return iterator pointing to the end of queue;
push_front()	Insert an element to the head of queue;
push_back()	Insert an element to the end of queue;
pop_front()	Delete the element at the head of queue;
pop_back()	Delete the element at the end of queue;
insert()	Insert elements to a random position;
erase()	Delete element at specified position;
at()	Return a reference of an element;
rend()	Return iterator of the converse end of queue;
rbegin()	Return iterator of the converse head of queue;

In addition, it is easy to access elements with iterator variables. Generally, operation of deque is convenient.

IV. WORKING PRINCIPLE

In visual C++ , the longest built in integer type is unsigned _int64, so it is right to define n as 32. The maximum digital symbol in 2^{32} based number system is 0xffffffff in hexadecimal which can be seated in an unsigned long variable. The sum or product of two digital will be settled in an unsigned _int64 variable because it may overstep the limit of unsigned long. In 2^{32} based number system, a number with K binary bits is [K/n] digitals long (figure 1.), and the calculation of unlimited integer has been turned to operations of deque elements and calculations of unsigned long variables.

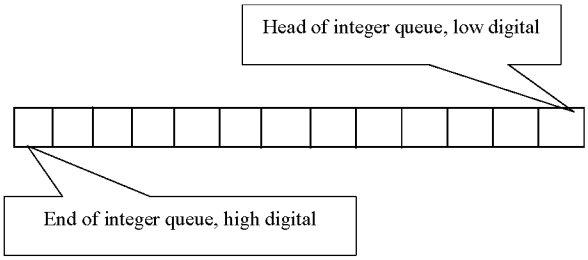


Figure 1. Digital queues of unlimited integer

In unlimited integer class, each element of integer queue is a digital of 2^n based number system; the following is the definition of data structure:

```
deque <unsigned long> UN_Integer_Deque;;
    //integer deque, the head is low position digital
bool   NegativeFlag;
    //number sign, true represents negative number
```

It is necessary to implement the arithmetic algorithm of this class. Addition of two unlimited integers can be switched to addition or subtraction of two unlimited integers with the same NegativeFlag, multiplication and division can be transformed into operation of NegativeFlag and operation of deques. Underside, the addition, subtraction, multiplication and division of unlimited integers are discussed.

As manual work, after decimal point position aligned, the addition of two numbers is performed by a loop from head of integer queues to end of integer queues. In case of adding unlimited number A and B, assuming that unlimited number object R is used to save the result, integer i represents the relative position of a digital, a_i, b_i and r_i are their corresponding

digital in position i, unsigned long variable C(0 or 1) is the carry from digital position i-1, unsigned _int64 S is the depositary of the sum of $a_i + b_i$+C, then the lower half part of S is r_i and the high half part of S is the next carry C. The carry from the end of decimal queue will attend the addition of digitals in the head of integer queue.

The subtraction of two unlimited integers is finally founded on the operation that the bigger number in absolute subtract the smaller one. The loop is performed as addition. For instance, A-B is needed to be calculated and the result will be left in R, C (0 or 1) is the borrow bit from the higher digital, and unsigned _int64 S is used to save the result of one step, the following is the pseudo code of subtraction of one digital r_i:

S= b_i+C
If S<= a_i
Then $r_i = a_i$ -S, C=0
Else $r_i = a_i + 2^{32}$-S, C=1

Multiplication can be performed by digital multiplication, shift and addition of middle results. It is necessary to save the middle result of one digital of multiplicator multiplies the multiplicand unlimited number, and the middle result will be shifted properly and added to the final result. The shift left operation is actually inserting an element 0 to the head of integer queue. The shift right operation is deleting the head element of integer queue; the integer queue has at least one element even if the integer is 0.

Division is relatively complex because it must test quotients each time. The quotient test procedure must recur to multiplication and subtraction of unlimited integers. For example, there are unlimited integers A, B, C and D, A is the dividend, B (B!=0 and A>B) is the divisor, C will be the quotient and D will be the remainder, the division arithmetic can be described as following steps:

1, Copy the highest part of A with the length of B to D, and then insert an element 0 to the end of integer queue of D;

2, If D<B, then insert an element 0 to the lower position of C, it means this quotient digital is 0;

If D=B, then insert an element 1 to the lower position of C, it means this quotient digital is 1, and do D=D-B;

If D>B, then try to find the quotient as following steps:

(1) Copying the highest two digital of D to variable d2 (unsigned _int64), copy the highest one digital of B to variable b (unsigned _int64), assuming q (unsigned _int64) is the tried quotient;

(2) Minimum of q is d2/(b+1), maximum of q is min (d2/b, 2^{32}-1) , by dichotomy, it is possible to find one suitable q settle for D<=B*(q+1) and D>=B*q. At this time, the right quotient must be q or q+1, if D<B*(q+1) then the quotient is q, else the quotient is q+1. The quotient would be inserted to the lower position of C;

(3) Subtract the product of B and the quotient from D. Temporality the highest digital of D must be 0, because D<B, and the length of D is 1 digital more than B;

3, Insert the next digital from A to the lower position of D, and delete the highest element of D (it is 0), and repeat step 2 until all elements of A has been treated.

4, D is the remainder.

All arithmetic calculations of unlimited integers can be founded on the four algorithms upon.

V. PERFORMANCE ANALYSIS

Because C++ is an object oriented programming language, it is convenient to define unlimited numeric class as a new class, and implement the conversion functions with other built-in data types such as int, long, _int64 and so on. It is necessary to realize in-out and storage operations of kinds of formats of the class. This class will be used as easy as built-in data types.

The unlimited integer class is obviously better than other built-in types in expression capability, because the length of the integer part is unlimited and automatically scalable.

Compared with the big integer solution based on array in predefined length, the unlimited number class has an obvious superiority that its length can change automatically to meet the actual need, and it is more efficient in memory using.

On the other hand, the relative weakness of unlimited integer class is the lower calculation speed, because the element operation speed of deque is lower than array. Table I shows the result of one experiment, it lists some comparable time of solutions based on deque and array to perform 1024 bit integer calculation on the same computer(Core 2 Quad Q6600/2400MHz/2048MB).

About calculation (X^Y mod Z), the unlimited integer class cost about 20s to perform 2048 bit calculation, and cost about 35s to perform 4096 bit calculation.

It is gratifying to see that the calculation time on 1024 bit numbers of unlimited integer class is acceptable. The speed of unlimited integer class will increase availably while deque's operation speed improved.

TABLE I
CALCULATION TIME OF 1024 BITS INTEGER

	Deque	Array
5000 additions	60ms	1ms
X^Y mod Z	10-15s	1-8s

(ms:millisecond; s:second)

This class is valuable to be a member of base library of high-level programming languages, and it can be one basement part of cryptogram class library.

REFERENCES

[1] JF. Liu, "Radix's selection of big integral's calculating accurately", Journal of East China Jiao tong University, vol.24 no.2, April 2007.

[2] YD. Liu, "Implementation for operation of big integer based on array", Computer and Modernization, March 2008.

[3] J. Yang, KM. Tang, XQ. Yang, "Construction and implement of characterizing large integer numerical calculation system", Journal of Yangzhou University (Natural Science Edition), vol.3 no.4, November 2000.

[4] P. Zhao, DD. Zhang, ZF. Tian, "Large integer multiplication in SIMD-LA", Microcomputer information, vol.22, December 2006.

[5] F. Gao, YZ. Wang, LQ. Sang, R. Shi, "The implementation of multiplication and division of big integer on PC machines", Journal of Logistical Engineering University, January 2007.

[6] CS. Ying, XL. Zhou, "Data structure and algorithm selection of big integer multiplication", Journal of Changchun University of Technology (Natural Science Edition), vol.29 no.2, April 2008.

[7] Y. Wang, L. Zhou, YK. Wang, B. Zhou, "The software realization for the modular arithmetic of large integer", China Information Security, February 2005.

[8] WP. Xie, DZ. Chen, "The machine testing and software realizing of primality of large number", Journal of Nanhua University (Science and Technology), vol.20 no.2, June 2006.

[9] Y. Shi, S. Yao, "Realization of arithmetic operation of the large integer", Journal of Anqing Teachers College (Natural Science), vol.10 no.2, May 2004.

[10] GL. Zhu, FM. Li, "Exact solution of multiplication of large integers", Journal of North China Institute of Water Conservancy and Hydroelectric Power, vol.28 no.3, June 2007.

[11] ZH. Wang, "A novel calculation method for the factorization of large integral numbers and the decoding of the RSA Cryptosystem", Acta Scientiarum Naturalium Universitatis Sunyatseni, vol.42 no.5, May 2003.

[12] KB. Dai, JH. Chen, "Optimization of the quadratic sieve in large integer factorization", Journal of Math (PRC), vol.25 no.6, June 2005.

[13] C. Ling, L. Mai, "Operation and performance comparisons of big integers based on two different memories", Journal of Anqing Teachers College (Natural Science), vol.9 no.1, February 2003.

[14] Bruce Schneier, Applied cryptography, John Wiley & Sons, 1996.

[15] Wang YB, Xue T, Applied cryptography, China Machine Press, 2003.

[16] Lv FX, C++ programming, China Publishing House of Electronics Industry, February 2002.

[17] Richard Johnsonbaugh, Object oriented programming in C++, China Machine Press, March 2005.